

Tailwind CSS lecture

Course Roadmap

- Lecture 1: What is Tailwind + installation/setup + utility-first concept
- Lecture 2: Colors + Text + Font + Spacing
- Lecture 3: Width + Height + Borders + Radius + Shadows
- Lecture 4: Flexbox
- Lecture 5: Grid
- Lecture 6: Responsive Design
- Lecture 7: Hover, Focus & other states
- Lecture 8: Positioning
- Lecture 9: Tailwind with React
- Lecture 10: Build a complete responsive project of weather app using Firebase Authentication using Weather API key given task based installation with Vite React Tailwind

Lecture 1 — Introduction, Setup & Utility-First Concept

Learning Objectives

- What is Tailwind CSS?
- What does utility-first mean?
- Why use Tailwind?
- Tailwind vs normal CSS?

Important Questions & Answers

Q: What is Tailwind CSS?

Answer: Tailwind CSS is a utility-first CSS framework that provides small, reusable utility classes for styling HTML and React elements.

Q: What does utility-first mean?

Answer: Instead of creating a custom CSS class and writing many properties inside it, we compose small utility classes directly on an element.

Q: Why use Tailwind?

Answer: It speeds up UI development, makes responsive design convenient, reduces custom CSS, and works well with component-based frameworks such as React.

Q: Tailwind vs normal CSS?

Answer: Normal CSS commonly separates class definitions from markup. Tailwind lets developers compose styling from predefined utilities directly in markup.

Coding Example

```
npm create vite@latest tailwind-class -- --template react
cd tailwind-class
npm install
```

```
npm install tailwindcss @tailwindcss/vite
```

```
// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})

// src/index.css
@import "tailwindcss";

// src/App.jsx
function App() {
  return (
    <main className="min-h-screen p-8">
      <h1 className="text-4xl font-bold text-blue-600">Hello Tailwind</h1>
      <p className="mt-3 text-gray-600">My first Tailwind UI.</p>
      <button className="mt-5 rounded-lg bg-blue-600 px-5 py-2 text-white hover:bg-blue-700">
        Start
      </button>
    </main>
  )
}
export default App
```

Practice Problem Statement

Create a Vite React project and install Tailwind CSS using the current Tailwind/Vite setup documented for your installed Tailwind version. Students should follow the official installation command/configuration for the version used in class.

Create a heading, paragraph and button using only Tailwind utilities.

Practice Coding Solution

```
npm create vite@latest tailwind-class -- --template react
cd tailwind-class
npm install
npm install tailwindcss @tailwindcss/vite

// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})

// src/index.css
@import "tailwindcss";

// src/App.jsx
function App() {
  return (
    <main className="min-h-screen p-8">
      <h1 className="text-4xl font-bold text-blue-600">Hello Tailwind</h1>
      <p className="mt-3 text-gray-600">My first Tailwind UI.</p>
      <button className="mt-5 rounded-lg bg-blue-600 px-5 py-2 text-white hover:bg-blue-700">
```

```

      Start
    </button>
  </main>
)
}
export default App

```

Lecture 2 — Colors, Text, Fonts & Spacing

Learning Objectives

- How do text colors work?
- How do background colors work?
- What is p-4?
- What is px-4 and py-4?
- What is mb-4?
- How do you make text bold?

Important Questions & Answers

Q: How do text colors work?

Answer: Classes such as text-blue-600 apply text color.

Q: How do background colors work?

Answer: Classes beginning with bg-, such as bg-blue-600, set background color.

Q: What is p-4?

Answer: It applies padding on all sides using Tailwind's spacing scale.

Q: What is px-4 and py-4?

Answer: px-4 applies horizontal padding; py-4 applies vertical padding.

Q: What is mb-4?

Answer: It applies margin-bottom.

Q: How do you make text bold?

Answer: Use a font-weight utility such as font-bold or font-semibold.

Coding Example

```

<div className="max-w-sm rounded-xl bg-white p-6 shadow">
  <h2 className="text-2xl font-bold text-gray-900">Student Card</h2>
  <p className="mt-2 text-gray-600">Learning Tailwind CSS utilities.</p>
  <button className="mt-4 rounded-lg bg-blue-600 px-5 py-2 font-semibold text-white">
    View Profile
  </button>
</div>

```

Practice Problem Statement

Create a student card with a title, description and button. Use at least two colors, three text utilities, padding, margin and gap.

Practice Coding Solution

```
<div className="max-w-sm rounded-xl bg-white p-6 shadow">
  <h2 className="text-2xl font-bold text-gray-900">Student Card</h2>
  <p className="mt-2 text-gray-600">Learning Tailwind CSS utilities.</p>
  <button className="mt-4 rounded-lg bg-blue-600 px-5 py-2 font-semibold text-white">
    View Profile
  </button>
</div>
```

Lecture 3 — Width, Height, Borders, Radius & Shadows

Learning Objectives

- What is w-full?
- What is h-screen?
- How do you add a border?
- How do you round corners?
- What does shadow-lg do?

Important Questions & Answers

Q: What is w-full?

Answer: It makes an element take the available width of its parent.

Q: What is h-screen?

Answer: It gives the element a height corresponding to the viewport height.

Q: How do you add a border?

Answer: Use border, then optionally border-2, border-gray-300, etc.

Q: How do you round corners?

Answer: Use utilities such as rounded-md, rounded-xl or rounded-full.

Q: What does shadow-lg do?

Answer: It applies a larger predefined box shadow.

Coding Example

```
<div className="mx-auto max-w-md rounded-2xl border border-gray-200 bg-white p-6 shadow-lg">
  <div className="h-40 w-full rounded-xl bg-gray-200"></div>
  <h2 className="mt-4 text-2xl font-bold">Product Card</h2>
  <p className="mt-2 text-gray-600">A card using Tailwind dimensions and visual utilities.</p>
  <button className="mt-4 rounded-lg bg-black px-5 py-2 text-white">Buy Now</button>
</div>
```

Practice Problem Statement

Build a product/profile card with a fixed maximum width, border, rounded corners, shadow, image placeholder and button.

Practice Coding Solution

```
<div className="mx-auto max-w-md rounded-2xl border border-gray-200 bg-white p-6 shadow-lg">
  <div className="h-40 w-full rounded-xl bg-gray-200"></div>
  <h2 className="mt-4 text-2xl font-bold">Product Card</h2>
  <p className="mt-2 text-gray-600">A card using Tailwind dimensions and visual utilities.</p>
  <button className="mt-4 rounded-lg bg-black px-5 py-2 text-white">Buy Now</button>
</div>
```

Lecture 4 — Flexbox

Learning Objectives

- What does flex do?
- What does justify-center do?
- What does items-center do?
- What is flex-col?
- What does justify-between do?

Important Questions & Answers

Q: What does flex do?

Answer: It changes an element into a flex container.

Q: What does justify-center do?

Answer: It controls alignment along the main axis and centers flex items.

Q: What does items-center do?

Answer: It centers items along the cross axis.

Q: What is flex-col?

Answer: It changes the main axis to a vertical column.

Q: What does justify-between do?

Answer: It distributes available space between flex items.

Coding Example

```
<nav className="flex flex-col gap-4 p-5 md:flex-row md:items-center md:justify-between">
  <h1 className="text-xl font-bold">MyApp</h1>
  <div className="flex gap-5">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
</nav>
```

Practice Problem Statement

Create a navbar with a logo on the left and three links on the right. On a small screen, arrange the content vertically.

Practice Coding Solution

```
<nav className="flex flex-col gap-4 p-5 md:flex-row md:items-center md:justify-between">
  <h1 className="text-xl font-bold">MyApp</h1>
  <div className="flex gap-5">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
</nav>
```

Lecture 5 — CSS Grid

Learning Objectives

- What does grid do?
- What does grid-cols-3 mean?
- What is gap-4?
- When should you use Grid instead of Flexbox?

Important Questions & Answers

Q: What does grid do?

Answer: It changes an element into a CSS Grid container.

Q: What does grid-cols-3 mean?

Answer: It creates three equal grid columns.

Q: What is gap-4?

Answer: It adds consistent spacing between grid items.

Q: When should you use Grid instead of Flexbox?

Answer: Grid is especially useful for two-dimensional layouts with rows and columns; Flexbox is mainly one-dimensional.

Coding Example

```
<div className="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
  <div className="rounded-xl bg-white p-5 shadow">Card 1</div>
  <div className="rounded-xl bg-white p-5 shadow">Card 2</div>
  <div className="rounded-xl bg-white p-5 shadow">Card 3</div>
  <div className="rounded-xl bg-white p-5 shadow">Card 4</div>
  <div className="rounded-xl bg-white p-5 shadow">Card 5</div>
  <div className="rounded-xl bg-white p-5 shadow">Card 6</div>
</div>
```

Practice Problem Statement

Create a dashboard containing six cards. Use one column on small screens, two on medium screens and three on large screens.

Practice Coding Solution

```
<div className="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
  <div className="rounded-xl bg-white p-5 shadow">Card 1</div>
```

```

<div className="rounded-xl bg-white p-5 shadow">Card 2</div>
<div className="rounded-xl bg-white p-5 shadow">Card 3</div>
<div className="rounded-xl bg-white p-5 shadow">Card 4</div>
<div className="rounded-xl bg-white p-5 shadow">Card 5</div>
<div className="rounded-xl bg-white p-5 shadow">Card 6</div>
</div>

```

Lecture 6 — Responsive Design

Learning Objectives

- What is responsive design?
- What is mobile-first design?
- What are sm, md, lg, xl and 2xl?
- How do you make a layout change from column to row?

Important Questions & Answers

Q: What is responsive design?

Answer: Responsive design makes a UI adapt to different screen sizes.

Q: What is mobile-first design?

Answer: Start with the base/mobile layout and add larger-screen behavior using responsive prefixes.

Q: What are sm, md, lg, xl and 2xl?

Answer: They are responsive breakpoint prefixes that apply a utility at the corresponding minimum breakpoint.

Q: How do you make a layout change from column to row?

Answer: Use a base class such as flex-col and a responsive class such as md:flex-row.

Coding Example

```

<section className="p-4 md:p-8">
  <h1 className="text-2xl font-bold md:text-4xl">Responsive Dashboard</h1>
  <div className="mt-6 grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
    <div className="rounded-xl p-5 shadow">One</div>
    <div className="rounded-xl p-5 shadow">Two</div>
    <div className="rounded-xl p-5 shadow">Three</div>
  </div>
</section>

```

Practice Problem Statement

Make a three-card layout responsive: one column on mobile, two on tablet, three on desktop. Also make heading sizes responsive.

Practice Coding Solution

```

<section className="p-4 md:p-8">
  <h1 className="text-2xl font-bold md:text-4xl">Responsive Dashboard</h1>
  <div className="mt-6 grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
    <div className="rounded-xl p-5 shadow">One</div>
    <div className="rounded-xl p-5 shadow">Two</div>
    <div className="rounded-xl p-5 shadow">Three</div>
  </div>

```

```
</div>
</section>
```

Lecture 7 — Hover, Focus & Other States

Learning Objectives

- How does hover work?
- How does focus work?
- What is disabled:?
- What are transitions used for?

Important Questions & Answers

Q: How does hover work?

Answer: Prefix a utility with hover:, for example hover:bg-blue-700.

Q: How does focus work?

Answer: Use focus: utilities to style an element when it receives focus.

Q: What is disabled:?

Answer: It applies styles when a form control is disabled.

Q: What are transitions used for?

Answer: They make changes between states visually smoother.

Coding Example

```
<form className="max-w-sm space-y-4">
  <input
    className="w-full rounded-lg border px-4 py-2 outline-none focus:border-blue-500 focus:ring-2
focus:ring-blue-200"
    placeholder="Email"
  />
  <input
    type="password"
    className="w-full rounded-lg border px-4 py-2 outline-none focus:border-blue-500 focus:ring-2
focus:ring-blue-200"
    placeholder="Password"
  />
  <button className="w-full rounded-lg bg-blue-600 py-2 text-white transition hover:bg-blue-700">
    Login
  </button>
</form>
```

Practice Problem Statement

Create a login form with inputs that show a visible focus state and a button with hover and transition effects.

Practice Coding Solution

```
<form className="max-w-sm space-y-4">
  <input
    className="w-full rounded-lg border px-4 py-2 outline-none focus:border-blue-500 focus:ring-2
focus:ring-blue-200"
```



```

        placeholder="Email"
    />
    <input
        type="password"
        className="w-full rounded-lg border px-4 py-2 outline-none focus:border-blue-500 focus:ring-2
focus:ring-blue-200"
        placeholder="Password"
    />
    <button className="w-full rounded-lg bg-blue-600 py-2 text-white transition hover:bg-blue-700">
        Login
    </button>
</form>

```

Lecture 8 — Positioning

Learning Objectives

- What is relative?
- What is absolute?
- What is fixed?
- What is sticky?
- What does z-10 do?

Important Questions & Answers

Q: What is relative?

Answer: It establishes a positioning context and allows an element to be positioned relative to its normal position.

Q: What is absolute?

Answer: It positions an element relative to its nearest positioned ancestor.

Q: What is fixed?

Answer: It positions an element relative to the viewport.

Q: What is sticky?

Answer: It allows an element to behave normally until a scroll threshold is reached.

Q: What does z-10 do?

Answer: It controls stacking order when positioning is relevant.

Coding Example

```

<div className="relative w-48">
  <div className="h-48 rounded-xl bg-gray-200"></div>
  <span className="absolute right-2 top-2 rounded-full bg-red-500 px-2 py-1 text-xs text-white">
    New
  </span>
</div>

```

Practice Problem Statement

Create a notification badge positioned at the top-right corner of a profile image/card.

Practice Coding Solution

```
<div className="relative w-48">
  <div className="h-48 rounded-xl bg-gray-200"></div>
  <span className="absolute right-2 top-2 rounded-full bg-red-500 px-2 py-1 text-xs text-white">
    New
  </span>
</div>
```

Lecture 9 — Tailwind with React

Learning Objectives

- How is Tailwind used in React?
- What is a React component?
- How can props help with Tailwind?
- Why create reusable components?

Important Questions & Answers

Q: How is Tailwind used in React?

Answer: Use Tailwind utility classes inside the className attribute.

Q: What is a React component?

Answer: A reusable UI unit that returns JSX.

Q: How can props help with Tailwind?

Answer: Props can provide content or state that changes the UI while the component keeps reusable styling.

Q: Why create reusable components?

Answer: They reduce duplication and make larger applications easier to maintain.

Coding Example

```
function Button({ children }) {
  return (
    <button className="rounded-lg bg-blue-600 px-5 py-2 text-white hover:bg-blue-700">
      {children}
    </button>
  )
}

function Card({ title, description }) {
  return (
    <div className="rounded-xl bg-white p-5 shadow">
      <h2 className="text-xl font-bold">{title}</h2>
      <p className="mt-2 text-gray-600">{description}</p>
      <div className="mt-4"><Button>View</Button></div>
    </div>
  )
}

export default function App() {
  const cards = [
    { title: "React", description: "Component-based UI." },
  ]
}
```

```

    { title: "Tailwind", description: "Utility-first styling." },
    { title: "Firebase", description: "Authentication and services." },
  ]

  return (
    <main className="grid gap-4 p-6 md:grid-cols-3">
      {cards.map((card) => (
        <Card key={card.title} {...card} />
      ))}
    </main>
  )
}

```

Practice Problem Statement

Create a reusable Button component and a reusable Card component. Render multiple cards from an array.

Practice Coding Solution

```

function Button({ children }) {
  return (
    <button className="rounded-lg bg-blue-600 px-5 py-2 text-white hover:bg-blue-700">
      {children}
    </button>
  )
}

function Card({ title, description }) {
  return (
    <div className="rounded-xl bg-white p-5 shadow">
      <h2 className="text-xl font-bold">{title}</h2>
      <p className="mt-2 text-gray-600">{description}</p>
      <div className="mt-4"><Button>View</Button></div>
    </div>
  )
}

export default function App() {
  const cards = [
    { title: "React", description: "Component-based UI." },
    { title: "Tailwind", description: "Utility-first styling." },
    { title: "Firebase", description: "Authentication and services." },
  ]

  return (
    <main className="grid gap-4 p-6 md:grid-cols-3">
      {cards.map((card) => (
        <Card key={card.title} {...card} />
      ))}
    </main>
  )
}

```

Lecture 10 — Final Project: Weather Application

Students build a complete responsive Weather Application using React, Vite, Tailwind CSS, Firebase Authentication, and the provided Weather API key.

Project Objective

Students will develop a responsive weather application where users can register, login, logout, search for a city, fetch weather data from a Weather API, display current weather and additional weather information, and handle loading and error states.

Required Technologies

- React
- Vite
- Tailwind CSS
- Firebase Authentication
- Weather API
- JavaScript
- HTML
- CSS

Project Flow

```
Weather App
|
+-- Register
|
+-- Login
    |
    +-- Authenticated
        |
        +-- Weather Dashboard
            |
            +-- Search City
            |
            +-- Weather API
            |
            +-- Weather Results
```

Pages

- Login Page: email, password, login button, register link, error message and loading state.
- Register Page: name, email, password and confirm password.
- Weather Dashboard: search city, current weather, temperature, condition, icon, humidity, wind speed and feels-like temperature, plus logout.

Required Components

```
src/
├── components/
│   ├── Navbar.jsx
│   ├── SearchBar.jsx
│   ├── WeatherCard.jsx
│   ├── WeatherDetails.jsx
│   ├── Loading.jsx
│   └── ErrorMessage.jsx
├── pages/
│   ├── Login.jsx
│   ├── Register.jsx
│   └── Dashboard.jsx
├── configuration/
│   ├── firebase.js
│   └── weatherApi.js
└── App.jsx
```

```
|— main.jsx
|— index.css
```

Final Project Problem Statement

Build a responsive Weather Application using React, Vite, Tailwind CSS, Firebase Authentication, and the provided Weather API key. Users must register and login through Firebase, access the weather dashboard only after authentication, search for a city, fetch live weather data, and see a responsive weather interface. The application must handle loading, invalid-city, and API-error states.

Authentication Task

```
Register
Email + Password
↓
Firebase Authentication
↓
Account Created
↓
Login

Login
Email + Password
↓
Firebase Authentication
↓
Successful Login
↓
Weather Dashboard

Logout
Logout Button
↓
Firebase Sign Out
↓
Login Page
```

Weather API Task

Students should use the city entered by the user to make the Weather API request. The weather result must come from the API rather than being hard-coded.

```
User enters: Karachi
↓
Weather API Request
↓
Weather Data
↓
React State
↓
WeatherCard
↓
Tailwind UI
```

Environment Variables

For Vite, students can keep configuration values in environment variables. A browser-exposed VITE_* variable is not a secure secret; it is bundled for client-side use. Never treat a frontend API key as a private server secret.

```
# .env
VITE_WEATHER_API_KEY=your_api_key
```

```
# JavaScript
const apiKey = import.meta.env.VITE_WEATHER_API_KEY
```

Final Project Requirements

- Create Register page.
- Create Login page.
- Register users with Firebase Authentication.
- Login users with Firebase Authentication.
- Implement Logout.
- Prevent unauthenticated users from accessing the Dashboard.
- Add city search.
- Fetch weather from the provided API.
- Display city name.
- Display temperature.
- Display weather condition.
- Display weather icon.
- Display humidity.
- Display wind speed.
- Display feels-like temperature.
- Handle invalid city.
- Handle API errors.
- Show loading state while fetching data.
- Use colors, typography, padding/margin, Flexbox, Grid, borders, radius, shadows, hover states, focus states, responsive breakpoints and positioning.

Final Project Grading Rubric

Component	Marks
Vite + React setup	5
Tailwind implementation	15
Firebase Register/Login	15
Logout & authentication flow	10
Weather API integration	15
Weather UI	10
Responsive design	10
Error/loading handling	5
React component structure	5
Overall UI/UX	10
Total	100

Final Project Starter Code Structure

```
// App.jsx – suggested flow
import { useEffect, useState } from "react"

export default function App() {
  const [user, setUser] = useState(null)

  // Connect Firebase auth state here.
  // Render Login/Register when user is not authenticated.
  // Render Dashboard when user is authenticated.

  return (
    <main className="min-h-screen bg-slate-100">
      { /* Authentication pages or Dashboard */ }
    </main>
  )
}
```

```
    </main>
  )
}
```

Student Submission Checklist

- Vite React project runs successfully.
- Tailwind CSS is installed and working.
- Register works with Firebase.
- Login works with Firebase.
- Logout works.
- Unauthenticated users cannot access the weather dashboard.
- Weather API key is configured.
- City search works.
- Weather data is displayed dynamically.
- Loading and error states work.
- UI is responsive on mobile, tablet and desktop.
- Components are separated logically.
- Code is clean and readable.